



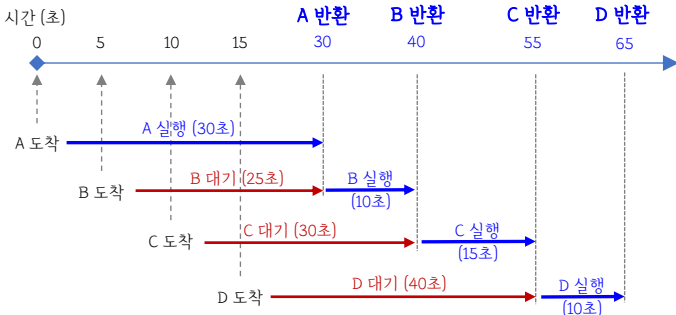
■ 프로세스 스케줄링 : 여러 프로세스의 처리 순서를 결정하는 기법

▶ 비선점형 : 기존 프로세서가 점유하고 있는 CPU를 빼앗을 수 없음
→ 낮은 처리율 / 적은 오버헤드

[예시] 아래 표의 프로세스 별 도착/실행 시간을 참조하여 각각의 스케줄링 적용하기

프로세스	도착시간	실행시간
A	0	30
B	5	10
C	10	15
D	15	10

① FCFS (Firs Come Firs Serve) : 프로세스 도착 순서에 따라 처리 순서 결정 (=FIFO)



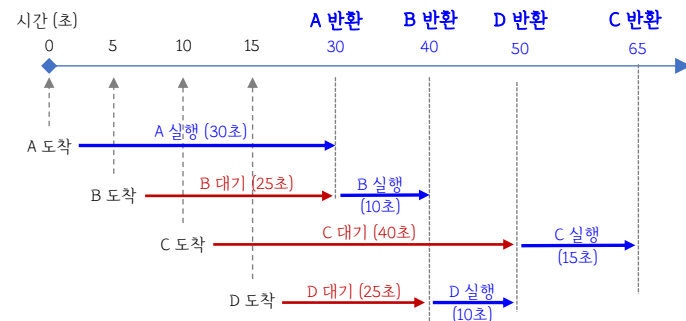
[FCFS 스케줄링]

	대기시간	실행시간	반환시간 (대기 + 실행)
A	0	30	30
B	25	10	35
C	30	15	45
D	40	10	50

평균 반환 시간
= (30+35+45+50) / 4 = 40

② SJF (Shortest Job First) : 실행 가능한 프로세스 중 실행 시간이 짧은 것부터 처리
A(30) → B(10) → D(10) → C(15)

A 프로세스가 완료된 시점에서 나머지 3개의 프로세스는 모두 도착하여 실행 가능함
이때, 실행 시간이 가장 짧으면서 먼저 도착한 B가 실행되고,
B가 완료되면 나머지 C, D 중에서 실행 시간이 짧은 D가 먼저 실행. 이후 C 실행



[SJF 스케줄링]

	대기시간	실행시간	반환시간 (대기 + 실행)
A	0	30	30
B	25	10	35
C	40	15	55
D	25	10	35

평균 반환 시간
= (30+35+55+35) / 4 = 38.75

③ HRN (Highest Response ratio Next) - 우선순위 = (대기시간 + 실행시간) / 실행시간
높은 순서대로 프로세스 진행

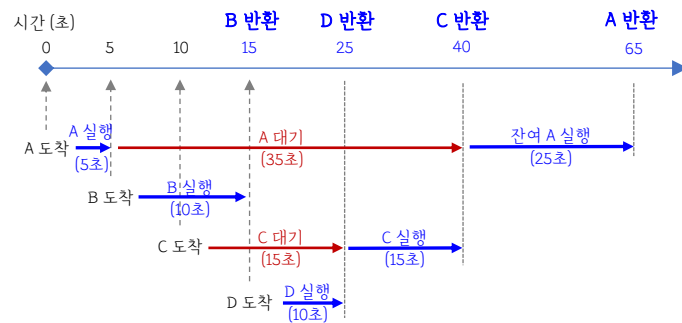
[HRN 스케줄링] 아래 표와 같이 대기/실행시간의 프로세스 별 우선순위 구하기
(모든 프로세스는 대기큐에 준비 중인 상태)

	대기시간	실행시간	우선순위
A	0	30	(0+30) / 30 = 1
B	5	10	(5+10) / 10 = 1.5
C	10	15	(10+15) / 15 = 1.66
D	15	10	(15+10) / 10 = 2.5

※ 우선순위 : D → C → B → A

▶ 선점형 : 운영체제가 실행 중인 프로세서로부터 CPU를 강제 빼앗음
→ 프로세서 별 CPU 처리 시간에 순서 조정으로 효율적 운영 가능 / 높은 오버헤드

④ SRT (Shortest Remaining Time) : 잔여 실행 시간이 가장 짧은 프로세스 우선 실행



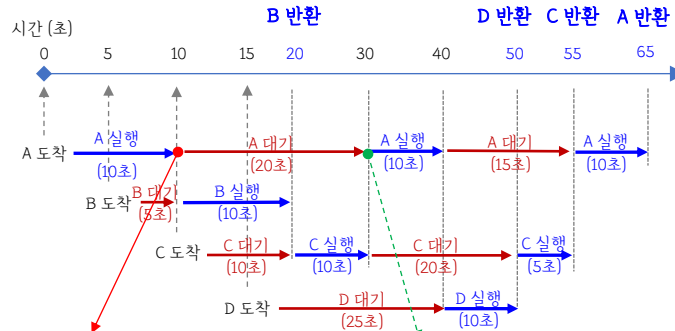
[SRT 스케줄링]

	대기시간	실행시간	반환시간 (대기 + 실행)
A	35	30	65
B	0	10	10
C	15	15	30
D	0	10	10

평균 반환 시간
= (65+10+30+10) / 4 = 28.75

⑤ RR (Round Robin) : 우선순위 없이 시간 단위로 CPU 할당 (공평한 자원 분배)
- 대기큐 도착 프로세스 순서대로 동일 시간 실행

[예시] 작업 시간 할당량(Time quantum)이 10인 RR 스케줄링 방식



A 실행이 끝났을 때 D는 아직 미도착 → 다음 대기큐에 A가 D보다 우선 진입했기에 두번째 A 먼저 실행 후 D 실행

[RR 스케줄링]

	대기시간	실행시간	반환시간 (대기 + 실행)
A	35	30	65
B	5	10	15
C	30	15	45
D	25	10	35

평균 반환 시간
= (65+15+45+35) / 4 = 40



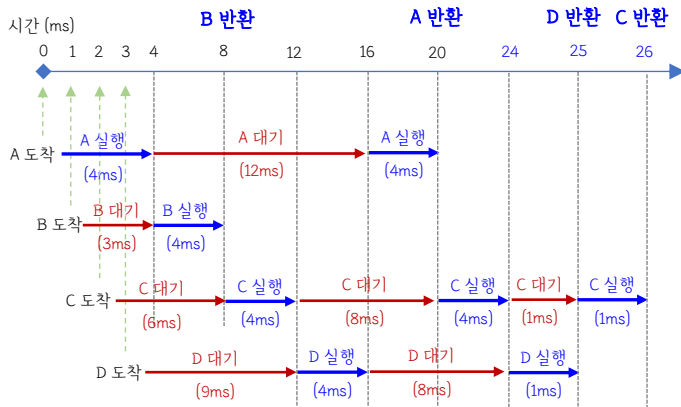
[기출예제] 25년도 2회차

아래 프로세스별 도착/실행 시간을 참고하여 평균 대기시간을 구하기

. 라운드로빈(RR) 스케줄링 방식 적용

. 시간 할당량 4 ms (스위칭 시간 무시)

프로세스	도착시간 (ms)	실행시간 (ms)
A	0	8
B	1	4
C	2	9
D	3	5



프로세스	도착시간	실행시간	반환시간	대기시간 (반환 - 도착 - 실행)
A	0	8	20	12
B	1	4	8	3
C	2	9	26	15
D	3	5	25	17

↓

평균 대기 시간

$$= (12+3+15+17) / 4 = 11.75 \text{ ms}$$